PIPE-Cypher: Automatic Enterprise Benchmark Generation for Text-to-Cypher Systems

Anonymous ACL submission

Abstract

Enterprises increasingly use property graphs and Cypher to analyze fraud, risk, identity, customer, and operational data. Public Text2Cypher benchmarks rarely reflect private schemas, terminology, access patterns, or difficulty distributions. We present PIPE-Cypher, a local-model pipeline for generating organization-specific natural-language-to-Cypher benchmarks. PIPE-Cypher combines schema introspection, reverse-query grounding, constrained Cypher generation, deterministic read-only and schema validation, execution feedback, repair, diversity controls, and LLM-judge review. The system is designed for industry settings where benchmark generation must avoid paid APIs, preserve data governance, and remain repeatable as graphs evolve.

1 Introduction

Property graphs encode enterprise relationships directly: account transfers, identity entitlements, access paths, customer interactions, and fraud rings. Cypher is a common query language for these graphs. As LLMs become natural-language interfaces for graph analytics, organizations need reliable benchmarks for natural-language-to-Cypher systems on their own schemas.

Static public benchmarks are insufficient for this setting. They cannot contain private labels, relationship types, categorical values, or domain-specific wording, and they do not track schema evolution. PIPE-Cypher addresses this gap by generating private, repeatable, execution-grounded NL-Cypher benchmarks.

2 Related Work

Recent Text2Cypher resources, including Mind the Query (Chauhan et al., 2025), SyntheT2C (Zhong et al., 2025), Auto-Cypher (Tiwari et al., 2025), and Text2Cypher (Ozsoy et al., 2025), show that high-quality Cypher data requires more than asking an LLM for query pairs. These works motivate schema grounding, execution checks, synthetic generation, verification, and complexity-aware evaluation. PIPE-Cypher differs by targeting private enterprise benchmark generation as the primary artifact: organizations bring their own graph, run local models, enforce Cypher constraints, and receive an executable benchmark with diversity and judge metadata.

Text-to-SQL benchmarks such as Spider 2.0 (Lei et al., 2024) and BIRD (Li et al., 2023) motivate realistic database tasks, execution-based evaluation, and enterprise workflow complexity. CIKM AutoQuery (Zheng et al., 2024) motivates strategy-level workload generation analysis and separating workload quality from model quality. LDBC FinBench (Qi et al., 2023) and SNB (P üroja et al., 2023) provide graph workloads suitable for industry-oriented evaluation.

For generated benchmark quality, PIPE-Cypher combines text-generation diversity diagnostics with text-to-query structure metrics. Distinct-n (Li et al., 2016) and self-BLEU-style redundancy checks (Zhu et al., 2018) capture lexical repetition, while schema coverage, query-signature diversity, structural feature rates, and normalized entropy over graph/category/difficulty cells capture properties that matter for executable Cypher benchmarks.

3 Method

PIPE-Cypher has five stages: schema profiling, template generation, reverse Cypher grounding, constrained Cypher generation and repair, deterministic validation and execution, and LLM-judge review. Deterministic checks enforce read-only safety, syntax shape, schema-visible labels and properties, explicit relationship types, observed relationship directions, schema-provided categorical values, and non-empty execution where required. A lightweight Cypher analyzer extracts return aliases, variables, labels, relationship observations, risky constructs, and rewrite skip reasons so normalization is auditable rather than a silent string edit. The direction gate interprets both outgoing and incoming Cypher arrow syntax before schema checking, and rejects undirected relationship patterns so directionality is an executable constraint rather than only a prompt instruction.

4 Implementation

The implementation is a Python package with Neo4j as the experimental backend. The paper frames the contribution around Cypher and property graphs rather than a single vendor. Local models are served through a vLLM/OpenAI-compatible endpoint on ds-serv6, using Qwen3.5 models for generation and judging.

The intended full-quality model is Qwen3.5-35B-A3B, but our June 1, 2026 live evidence uses the cached Qwen3.5-9B fallback. The 35B-A3B weights were staged on ds-serv6 under root-backed storage. A serving-capacity check estimated that the staged weights require four A5000 GPUs under our vLLM budget, while only one GPU was safely free in the live snapshot, so the reported generation and downstream results use the 9B fallback.

The built-in FinBench profile is grounded in the public datagen snapshot export and includes typed node and relationship properties. Live schema inspection also records low-cardinality string values as categorical constraints for prompting and validation. The generated Neo4j import script uses uniqueness constraints for nodes and creates, rather than merges, transaction relationships so repeated account-to-account events are preserved. The SNB reference profile is grounded in the official Neo4j/Cypher headers and read-query files.

For generation, PIPE-Cypher uses seeded workload templates with deterministic reverse-binding queries and a mixed mode that prepends proven workload seeds to LLM-proposed templates. Every run records accepted and rejected candidates for yield analysis. Retrieved few-shot examples replace graph-specific values with typed placeholders, preserving query structure while reducing tenant-value leakage and memorized entity reuse. A dependency-light value grounder adds typed prompt annotations for categorical values and reverse-bound entities, covering punctuation variants, possessives, plurals, synonyms, name partials, and small typos.

For LLM-judge review, the implementation slices schema context to the labels, relationship types, and properties used by the candidate query. This preserves validation against the full schema while keeping local 9B fallback-model prompts within context limits on larger schemas.

The deterministic Cypher layer borrows from production lessons in the BalkanID Cypher system: schema-only prompting, exact matching, relationship direction discipline, `RETURN DISTINCT`, reserved variable rejection, categorical values, required contextual return columns, fuzzy value annotations, placeholderized retrieval examples, and parser-aware rewrite boundaries. PIPE-Cypher records parser-style structure features and skips rewrites for risky constructs such as `UNION`, `CALL`, `UNWIND`, `WHERE EXISTS`, multiple `WHERE` clauses, or reserved variables.

We also estimate seed-template capacity before full generation. This check caught and fixed a scale blocker: reverse-binding execution was hard-capped to 10 rows, and no-slot negation/ranking seeds could not support full category targets. PIPE-Cypher now uses the configured binding limit and includes slotted negation and ranking seeds for both graphs.

5 Experiments

The generated benchmark contains 3,000 accepted examples: 2,000 from LDBC FinBench and 1,000 from LDBC SNB. Categories include simple retrieval, complex retrieval, simple aggregation, complex aggregation, boolean existence, negation/difference, path/temporal transaction, and ranking/top-k.

6 Results

The full live run produced 3,000 accepted examples from 4,777 candidates using local Qwen3.5-9B for generation and judging. Category-specific recovery top-ups filled the only under-target categories from the initial sequential run. Every exported example passed read-only, syntax, schema, execution, non-empty result, and judge gates.

The accepted full-run records were exported into a benchmark package with stable example identifiers, train/dev/test splits, result samples, gate metadata, aggregate statistics, and a manifest hash for artifact tracking. The export contains 3,000 accepted examples: 2,000 FinBench, 1,000 SNB, and

Gate	Evidence checked	Failure mode caught
Read-only safety	blocked Cypher write/admin tokens	destructive or operational queries
Schema validity	labels, relationship types, properties, categorical values	hallucinated graph vocabulary or values
Direction validity	observed relationship directions	reversed or invalid traversals
Cypher analysis and normalization	structure extraction, rewrite safety, `RETURN DISTINCT`	unsafe rewrites or duplicate/noisy rows
Question constraints	quoted values use exact literals	fuzzy matching of exact user values
Execution	query runs and returns rows	syntactic validity without answerability
LLM judge	ambiguity, semantic alignment, schema use	executable but semantically weak examples

Table 1: PIPE-Cypher candidate acceptance gates.

Graph	Target/category	Binding limit	Min seed capacity	All categories meet target
FinBench	250	300	300	Yes
SNB	125	200	200	Yes

Table 2: Built-in seed-template capacity after removing the reverse-binding 10-row cap and adding slotted negation/ranking seeds. Capacity is a theoretical scale-readiness check; final examples still must pass validation, execution, diversity, and judge gates.

Setting	Value
Accepted examples	3,000
Primary graph	LDBC FinBench, 2,000 examples
Secondary graph	LDBC SNB, 1,000 examples
Categories	8 balanced categories, 375 each
Generation/judge model	Local Qwen3.5-9B fallback
Execution backend	Neo4j Community, two live databases
Judge audit packet	80 sampled accepted/rejected pairs

Table 3: Full live experimental setup used for the generated benchmark artifact.

Graph	Candidates	Accepted	Acceptance	Categories at target
FinBench	3,404	2,000	0.588	8/8
SNB	1,373	1,000	0.728	8/8
Total	4,777	3,000	0.628	16/16

Table 4: Full live generation with local Qwen3.5-9B. Candidate counts include the initial sequential run and category-specific recovery top-ups.

Export artifact	Examples	FinBench	SNB	Train	Dev	Test
Live full benchmark	3,000	2,000	1,000	2,408	296	296

Table 5: Accepted live full benchmark package with stable IDs, gate metadata, result samples, statistics, and manifest hash `8bc79a53a06b291a`.

375 accepted examples in every planned category.

We report diversity as a diagnostic, not only as a design target. The full export has perfect category balance and near-perfect difficulty balance, but the Qwen3.5-9B fallback run still has low query-signature diversity because seeded graph-grounded templates are doing substantial work. This is useful evidence for industry deployment: the artifact is balanced and executable, while the appendix makes template concentration visible rather than hiding it.

The full-run failure taxonomy is concentrated in diversity/duplicate controls during recovery and empty execution results; only 2/4,777 candidates were schema-invalid after deterministic Cypher governance.

For judge calibration, the released artifacts include an 80-row audit packet sampled from accepted and rejected full-run candidates, with a 40/40 judge accept/reject split and coverage over both graphs and all eight categories. The calibration tooling tracks graph, category, difficulty, strategy, and judge-decision coverage, and reports agreement, precision/recall, specificity, balanced accuracy, and Cohen's kappa once human labels are filled. The current draft treats these labels as pending human review rather than as a generation gate.

Finally, we ran a downstream Text2Cypher evaluation using local Qwen3.5-9B on the exported full test split. The model saw schema text and the natural-language question, generated Cypher, and was evaluated by live execution against the corresponding FinBench or SNB database. The result verifies that the exported artifact can drive downstream model evaluation and gives a difficulty signal: the zero-shot 9B baseline solves simple aggregation examples but struggles on more operational categories.

7 Industry Use

PIPE-Cypher is intended to be rerun when an enterprise graph changes. The generated artifact records the schema snapshot, graph profile, model identifier, validation gates, execution samples, judge scores, difficulty features, and source run for every example. Long-running jobs can be monitored from JSONL records and recovered with category-specific top-up runs that reject questions accepted

Artifact property	Value
Categories	8 balanced categories
FinBench/category	250
SNB/category	125
Difficulty split	1,521 easy / 1,479 medium
Unique labels / rel. types	7 / 9
Read/syntax/schema/exec/judge gates	3,000/3,000

Table 6: Distribution and gate summary for the exported full benchmark artifact.

Split	Examples	Parse	Schema	Exec. success	Exec. acc.	Answer F1
Live full test	296	0.959	0.905	0.622	0.189	0.189

Table 7: Downstream Text2Cypher evaluation for local Qwen3.5-9B on the exported full benchmark test split.

in earlier runs. This design supports private benchmark refreshes without exposing schemas or values to paid APIs, while still leaving audit hooks for data owners to sample accepted and rejected examples.

8 Conclusion

PIPE-Cypher provides a practical path for enterprises to create private, executable, balanced NL-to-Cypher benchmarks. The pipeline combines schema introspection, constrained generation, deterministic validation, execution feedback, diversity control, and automated judge review.

Limitations

Execution validity does not guarantee semantic correctness. LLM judges can over-accept plausible but wrong examples, so the final study includes a small human audit for calibration. Generated artifacts may contain private schema details or sampled values, so organizations should apply normal data governance controls.

Ethics Statement

PIPE-Cypher is designed for private benchmark generation under local-model deployment constraints. Organizations using it should restrict benchmark artifacts to authorized users, review sampled values for sensitive content, and document whether judge calibration relied on human annotators.

References

Vashu Chauhan, Shobhit Raj, Shashank Mujumdar, Avirup Saha, and Anannay Jain. 2025. Mind the query: A benchmark dataset towards Text2Cypher task. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1890–1905, Suzhou, China. Association for Computational Linguistics.

Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin Su, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, Victor Zhong, Caiming Xiong, Ruoxi Sun, Qian Liu, Sida Wang, and Tao Yu. 2024. Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows. *Preprint*, arXiv:2411.07763.

Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Rongyu Cao, Ruiying Geng, Nan Huo, Xuanhe Zhou, Chenhao Ma, Guoliang Li, Kevin C. C. Chang, Fei Huang, Reynold Cheng, and Yongbin Li. 2023. Can LLM already serve as a database interface? a BIg bench for large-scale database grounded text-to-SQLs. In *Advances in Neural Information Processing Systems*.

Jiwei Li, Michel Galley, Chris Brockett, Jianfeng Gao, and Bill Dolan. 2016. A diversity-promoting objective function for neural conversation models. In *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 110–119, San Diego, California. Association for Computational Linguistics.

Makbule Gulcin Ozsoy, Leila Messallem, Jon Besga, and Gianandrea Minneci. 2025. Text2Cypher: Bridging natural language and graph databases. In *Proceedings of the Workshop on Generative AI and Knowledge Graphs*, pages 100–108, Abu Dhabi, UAE. International Committee on Computational Linguistics.

David P
üroja, Jack Waudby, Peter Boncz, and G
'abor Sz
'arnyas. 2023. The LDBC social network benchmark interactive workload v2: A transactional graph query benchmark with deep delete operations. *Preprint*, arXiv:2307.04820.

Shipeng Qi, Heng Lin, Zhihui Guo, G
'abor Sz
'arnyas, Bing Tong, Yan Zhou, Bin Yang, Jiansong Zhang, Zheng Wang, Youren Shen, Changyuan Wang, Parviz Peiravi, Henry Gabb, and Ben Steer. 2023. The LDBC financial benchmark. *Preprint*, arXiv:2306.15975.

Aman Tiwari, Shiva Krishna Reddy Malay, Vikas Yadav, Masoud Hashemi, and Sathwik Tejaswi Madhusudhan. 2025. Auto-cypher: Improving LLMs on cypher generation via LLM-supervised generation-verification framework. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2: Short Papers*, pages 623–640, Albuquerque, New Mexico. Association for Computational Linguistics.

Xiuwen Zheng, Arun Kumar, and Amarnath Gupta. 2024. Generating cross-model analytics workloads

Metric	Value
Question Distinct-1	0.041
Question Distinct-2	0.088
Question self-BLEU-2 (sampled)	0.826
Unique query-signature ratio	0.010
Top query-signature share	0.083
Category normalized entropy	1.000
Graph-category normalized entropy	0.980
Difficulty normalized entropy	1.000
Label coverage	0.412
Relationship-type coverage	0.375
Property-name coverage	0.407
Aggregation / negation / ordering rates	0.500 / 0.125 / 0.125

Table 8: Diversity diagnostics for the full exported benchmark. Distinct-n follows text-generation usage; self-BLEU is lower when questions are less redundant.

Failure bucket	Count	Share of rejected
Diversity/duplicate control	1,335	0.751
Empty result	374	0.210
Judge semantic reject	66	0.037
Schema invalid	2	0.001

Table 9: Failure taxonomy over full-run generation candidates before benchmark export. Accepted examples are excluded from the bucket shares.

Setting	Graph	Records	Accepted	Acceptance	Categories at target
Unconstrained LLM	FinBench	0	0	0.000	0/8
Unconstrained LLM	SNB	0	0	0.000	0/8
Reverse-only	FinBench	400	400	1.000	8/8
Reverse-only	SNB	402	400	0.995	8/8
Validators+repair	FinBench	400	400	1.000	8/8
Validators+repair	SNB	402	400	0.995	8/8
No retrieval	FinBench	429	400	0.932	8/8
No retrieval	SNB	402	400	0.995	8/8
No rewrite	FinBench	423	400	0.946	8/8
No rewrite	SNB	402	400	0.995	8/8
No LLM judge	FinBench	401	400	0.998	8/8
No LLM judge	SNB	406	400	0.985	8/8
Full PIPE-Cypher	FinBench	416	400	0.962	8/8
Full PIPE-Cypher	SNB	402	400	0.995	8/8

Table 10: Live target-50 ablation evidence with local Qwen3.5-9B. Each graph run targets 50 accepted examples per category.

Setting	Graph	Read-only	Syntax	Schema	Exec.	Judge
Unconstrained LLM	FinBench	0.000	0.000	0.000	0.000	0.000
Unconstrained LLM	SNB	0.000	0.000	0.000	0.000	0.000
Reverse-only	FinBench	1.000	1.000	1.000	1.000	1.000
Reverse-only	SNB	1.000	1.000	0.995	0.995	0.995
Validators+repair	FinBench	1.000	1.000	1.000	1.000	1.000
Validators+repair	SNB	1.000	1.000	0.995	0.995	0.995
No retrieval	FinBench	1.000	1.000	1.000	1.000	0.932
No retrieval	SNB	1.000	1.000	0.995	0.995	0.995
No rewrite	FinBench	1.000	1.000	1.000	1.000	0.946
No rewrite	SNB	1.000	1.000	0.995	0.995	0.995
No LLM judge	FinBench	1.000	1.000	1.000	1.000	0.998
No LLM judge	SNB	1.000	1.000	0.995	0.995	0.985
Full PIPE-Cypher	FinBench	1.000	1.000	1.000	1.000	0.962
Full PIPE-Cypher	SNB	1.000	1.000	0.995	0.995	0.995

Table 11: Quality-gate rates for the live target-50 ablation suite. Rates are computed over all generated records in each graph/setting.

using LLMs. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pages 4303–4307. ACM.

Zijie Zhong, Linqing Zhong, Zhaoze Sun, Qingyun Jin, Zengchang Qin, and Xiaofan Zhang. 2025. SyntheT2C: Generating synthetic data for fine-tuning large language models on the Text2Cypher task. In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 672–692, Abu Dhabi, UAE. Association for Computational Linguistics.

Yaoming Zhu, Sidi Lu, Lei Zheng, Jiaxian Guo, Weinan Zhang, Jun Wang, and Yong Yu. 2018. Texygen: A benchmarking platform for text generation models. In *The 41st International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1097–1100. ACM.

A Additional Results

B Claim–Evidence Map

This section maps the main paper claims to the strongest current evidence and to the remaining risks. This is intended as a reviewer-facing audit surface: claims with pending evidence remain marked as such rather than being folded into the main results.

1. **Claim 1.** PIPE-Cypher can generate a private, executable NL-to-Cypher benchmark over enterprise-style property graphs using local models.

   Evidence. The full Qwen3.5-9B fallback run exported 3,000 accepted examples: 2,000 FinBench and 1,000 SNB, with 375 examples in every planned workload category and all accepted examples passing read-only, syntax, schema, execution, non-empty-result, and judge gates.

   Key artifacts. benchmark export; manifest snapshot; paper table: benchmark export; paper table: full results

   Status. Supported by live 9B fallback evidence; 35B target run is blocked by current GPU capacity.

   Risk. Generation quality may change with the target 35B model or with private enterprise schemas beyond FinBench/SNB.

2. **Claim 2.** Cypher-specific governance makes generated benchmark candidates safer and more auditable than prompt-only generation.

   Evidence. The pipeline validates read-only safety, schema-visible labels/properties/relationships, observed relationship direction, categorical values, con-

Audit packet property	Value
Rows	80
Judge accept / reject	40 / 40
FinBench / SNB rows	48 / 32
Easy / medium rows	26 / 54
Labeled rows	0

Table 12: Post-hoc judge calibration packet coverage. Human labels are pending and are not used as a generation gate.

Metric	Point	95% CI	SE	N
Parse valid	0.959	[0.936, 0.980]	0.012	296
Schema valid	0.905	[0.872, 0.939]	0.017	296
Execution success	0.622	[0.564, 0.676]	0.028	296
Execution accuracy	0.189	[0.145, 0.233]	0.022	296
Answer F1	0.189	[0.149, 0.236]	0.023	296

Table 13: Bootstrap uncertainty for downstream Text2Cypher evaluation on the full exported test split. Intervals use row-level resampling with a fixed seed and are intended for appendix reporting.

textual return columns, parser-style structure, execution success, and LLM-judge review. In the full run, only 2 of 4,777 candidates were schema-invalid after deterministic governance, and all 3,000 exported examples passed the deterministic and judge gates.

Key artifacts. code: validator.py; code: cypher_parser.py; code: question_constraints.py; snapshot: failure taxonomy; +1 more

Status. Supported by code, tests, and full-run failure taxonomy.

Risk. Semantic correctness still depends on execution evidence and judge review; human audit labels are pending.

3. **Claim 3.** Reverse grounding and structured workload seeds are designed to improve reliability under local-model constraints.

   Evidence. The target-50 FinBench/SNB ablation suite completed 14/14 graph/variant cells, reached every category target for all non-empty/non-unconstrained runs, and passed the paper-readiness audit with logs, model IDs, code revision, run summaries, and gate-rate availability. Appendix-only tables and a figure report the audited target-50 evidence while the target-100 follow-up and seeded target-50 repeat continue for stronger final reliability and variance claims.

   Key artifacts. script: run live ablation suite; script: monitor remote ablation suite; script: monitor remote ablation queue; script: collect remote ablation suite; +8 more

   Status. Supported by audited target-50 appendix evidence; target-100 and seeded target-50 repeat runs are still pending for stronger final reliability claims.

   Risk. A single target-50 suite is the minimum publishable ablation scale, not the strongest possible evidence. If target-100 or repeated-seed runs complete, final claims should use those sensitivity results instead of relying only on target-50.

4. **Claim 4.** The benchmark controls category and difficulty balance while exposing residual diversity limits.

   Evidence. The full export has perfect category balance, planned 2:1 FinBench/SNB graph mix, and a near-even easy/medium split. Diversity diagnostics report Distinct-n, sampled self-BLEU-2, query-signature diversity, normalized entropy, schema coverage, and structural feature rates, making template concentration visible instead of hidden.

   Key artifacts. snapshot: diversity report; paper table: diversity; paper figure: diversity diagnostics; paper figure: full export distribution

   Status. Supported by full-export statistics and diversity diagnostics.

   Risk. Low query-signature diversity in the 9B fallback run remains a limitation and ablation target.

5. **Claim 5.** The generated benchmark is useful for downstream Text2Cypher evaluation.

   Evidence. A zero-shot local Qwen3.5-9B downstream evaluation over the 296-example full test split reports parse validity, schema validity, execution success, execution accuracy, answer F1, per-category breakdowns, and fixed-seed bootstrap uncertainty intervals. The model reaches 0.189 execution accuracy with a 95% bootstrap interval of [0.145, 0.233] and 0.622 execution success with interval [0.564, 0.676], showing the benchmark can discriminate easy aggregation from harder operational categories.

Key artifacts. downstream summary; snapshot: downstream uncertainty; research note: downstream evaluation; paper table: downstream evaluation; +3 more

Status. Supported by live downstream evaluation against FinBench/SNB databases, with bootstrap uncertainty computed from row-level outputs.

Risk. Only one downstream model has been evaluated so far.

6. **Claim 6.** Automated LLM-judge review can replace human-in-the-loop generation gates while still exposing judge risk.

   Evidence. The pipeline uses deterministic validation plus LLM-judge review as the generation gate. A post-hoc 80-row judge audit packet preserves 40 judge accepts, 40 judge rejects, both graphs, and all eight categories, and the analyzer reports agreement, precision/recall, specificity, balanced accuracy, and Cohen's kappa once human labels are filled.

   Key artifacts. code: judge.py; code: calibration.py; code: judge_audit_packet.py; snapshot: judge audit packet v2; +2 more

   Status. Pipeline gate is implemented; audit packet coverage is supported.

   Risk. Judge-human agreement is not yet known because human labels are still blank.

7. **Claim 7.** The intended 35B local generation/judge path is staged but not currently runnable under observed shared-GPU constraints.

   Evidence. Qwen3.5-35B-A3B is staged on ds-serv6, but the capacity checker reports 68,573 MiB of weights, four required A5000 GPUs under the conservative vLLM budget, and only one safe GPU in the latest snapshot.

   Key artifacts. script: check vllm capacity; research note: model availability; research note: qwen35b capacity snapshot 20260601; snapshot: qwen35b capacity 20260601 latest

   Status. Documented blocker; current paper results are explicitly reported as Qwen3.5-9B fallback evidence.

   Risk. A later 35B run may change yield, diversity, judge behavior, and downstream conclusions.

C Prompt Contracts

PIPE-Cypher treats prompts as versioned implementation artifacts. The list summarizes the prompt contracts used for generation, repair, judging, and downstream evaluation; hashes fingerprint the full prompt constants in the codebase.

1. **Template generation.** Stage: Workload proposal. SHA-256: `10b25b`.

   Contract. Schema-only labels, relationships, properties, and categorical values; Realistic enterprise analyst wording; At most two typed slots and JSON-only output

2. **Reverse binding.** Stage: Graph grounding. SHA-256: `48e949`.

   Contract. Read-only MATCH/WHERE/RETURN DISTINCT/LIMIT only; Slot variables named exactly as requested; Forward relationship directions from the schema

3. **Cypher generation.** Stage: Candidate query. SHA-256: `61c557`.

   Contract. Only schema-visible constructs and observed directions; RETURN DISTINCT for set returns and exact equality for quoted values; Context columns, categorical hints, placeholderized retrieval, and no writes

4. **Repair.** Stage: Validation feedback. SHA-256: `56c419`.

   Contract. Preserve question intent while fixing validation or execution issues; Keep query read-only and schema-grounded; Return only corrected Cypher

5. **LLM judge.** Stage: Quality gate. SHA-256: `073938`.

   Contract. Inputs include question, Cypher, schema slice, execution rows, and validation summary; Strict JSON scores for ambiguity, semantic alignment, schema use, and difficulty; Pass only useful, unambiguous enterprise benchmark examples

6. **Downstream Text2Cypher.** Stage: Model evaluation. SHA-256: `4c07ff`.

Contract. Read-only Cypher only; Schema-visible constructs and exact direction preservation; RETURN DISTINCT, count/ranking rules, and no explanations

D Representative Accepted Examples

The examples below are selected in stable identifier order from the tracked full-export snapshot, one per graph/category cell when available. They show the natural-language question, accepted Cypher, structural tags, gate status, and a bounded execution-result sample.

1. **FinBench / Boolean Existence / easy.** *Question:* Does account '187743809466009406' have any outgoing transfer?

```
MATCH (src:Account {accountId:
'187743809466009406'})
-[:TRANSFER_TO]->
(:Account)
RETURN DISTINCT COUNT(DISTINCT src)
> 0
AS HasOutgoingTransfer
```

   Structure: single_hop, aggregation.

   Relationships: TRANSFER_TO.

   Gates: RO/Syn/Schema/Exec/Judge.

   Result sample: {HasOutgoingTransfer: True}; observed rows: 1

2. **FinBench / Complex Aggregation / medium.** *Question:* What is the total transferred amount from accounts owned by person 'Gwar'?

```
MATCH (p:Person {personName:
'Gwar'})
-[:OWN_ACCOUNT]->
(src:Account)-[t:TRANSFER_TO]->
(:Account)
RETURN DISTINCT SUM(t.amount) AS
TotalTransferredAmount
```

   Structure: join_heavy, aggregation.

   Relationships: OWN_ACCOUNT, TRANSFER_TO.

   Gates: RO/Syn/Schema/Exec/Judge.

   Result sample: {TotalTransferredAmount: 14972318.41}; observed rows: 1

3. **FinBench / Complex Retrieval / easy.** *Question:* Which accounts received transfers from accounts owned by person 'Zof'?

```
MATCH (p:Person {personName:
'Zof'})
-[:OWN_ACCOUNT]-> (src:Account)
-[:TRANSFER_TO]-> (dst:Account)
RETURN DISTINCT dst.accountId AS
AccountId, dst.accountType AS
AccountType, dst.isBlocked AS
IsBlocked
LIMIT 300
```

   Structure: join_heavy, bounded_result.

   Relationships: OWN_ACCOUNT, TRANSFER_TO.

   Gates: RO/Syn/Schema/Exec/Judge.

   Result sample: {AccountId: 4687402787162554854, AccountType: credit card, IsBlocked: False}; observed rows: 3

4. **FinBench / Negation Difference / medium.** *Question:* Which accounts owned by person 'Kant' have not sent any transfers?

```
MATCH (p:Person {personName:
'Kant'})
-[:OWN_ACCOUNT]-> (a:Account)
WHERE NOT (a) -[:TRANSFER_TO]->
(:Account)
RETURN DISTINCT a.accountId AS
AccountId, a.accountType AS
AccountType,
a.isBlocked AS IsBlocked
LIMIT 300
```

   Structure: join_heavy, negation, bounded_result.

   Relationships: OWN_ACCOUNT, TRANSFER_TO.

   Gates: RO/Syn/Schema/Exec/Judge.

   Result sample: {AccountId: 4739757132830738341, AccountType: merchant account, IsBlocked: False}; observed rows: 1

5. **FinBench / Path Temporal / medium.** *Question:* Which accounts can receive money within two transfer hops from accounts owned by person 'Sossamon'?

```
MATCH (p:Person {personName:
'Sossamon'}) -[:OWN_ACCOUNT]->
(src:Account) -[:TRANSFER_TO*1..2]->
(dst:Account)
RETURN DISTINCT dst.accountId AS
AccountId, dst.accountType AS
AccountType, dst.isBlocked AS
IsBlocked
LIMIT 300
```

   Structure: join_heavy, path, bounded_result.

   Relationships: OWN_ACCOUNT, TRANSFER_TO.

   Gates: RO/Syn/Schema/Exec/Judge.

   Result sample: {AccountId: 4687402787162554854, AccountType: credit card, IsBlocked: False}; observed rows: 4

6. **FinBench / Ranking Topk / medium.** *Question:* For accounts owned by person 'Barry',

which account sent the highest total transfer amount?

```
MATCH (p:Person {personName:
'Barry'})
-[:OWN_ACCOUNT]->
(src:Account)-[t:TRANSFER_TO]->
(:Account)
WITH src, SUM(t.amount) AS
totalAmount
RETURN DISTINCT src.accountId AS
AccountId, src.accountType AS
AccountType, src.isBlocked AS
IsBlocked,
totalAmount
ORDER BY totalAmount DESC
LIMIT 1
```

Structure: join_heavy, aggregation, order_rank, bounded_result.

Relationships: OWN_ACCOUNT, TRANSFER_TO.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {AccountId: 4732438783436260772, AccountType: internet account, IsBlocked: False}; observed rows: 1

7. **FinBench / Simple Aggregation / easy.** *Question:* How many accounts are owned by person 'Kaewsuktae'?

```
MATCH (p:Person {personName:
'Kaewsuktae'}) -[:OWN_ACCOUNT]->
(a:Account)
RETURN DISTINCT COUNT(DISTINCT a) AS
AccountCount
```

Structure: single_hop, aggregation.

Relationships: OWN_ACCOUNT.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {AccountCount: 1}; observed rows: 1

8. **FinBench / Simple Retrieval / easy.** *Question:* Which accounts are owned by person 'Barry'?

```
MATCH (p:Person {personName:
'Barry'})
-[:OWN_ACCOUNT]-> (a:Account)
RETURN DISTINCT a.accountId AS
AccountId, a.accountType AS
AccountType,
a.isBlocked AS IsBlocked
LIMIT 300
```

Structure: single_hop, bounded_result.

Relationships: OWN_ACCOUNT.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {AccountId: 4732438783436260772, AccountType: internet account, IsBlocked: False}; observed rows: 1

9. **SNB / Boolean Existence / medium.** *Question:* Does person with id 6597069766828 like any post?

```
MATCH (p:Person {id:
6597069766828})
OPTIONAL MATCH (p) -[:LIKES]->
(post:Post)
RETURN DISTINCT COUNT(post) > 0 AS
LikesAnyPost
```

Structure: single_hop, aggregation, optional.

Relationships: LIKES.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {LikesAnyPost: True}; observed rows: 1

10. **SNB / Complex Aggregation / medium.** *Question:* How many distinct posts are in forums joined by person with id 6597069766845?

```
MATCH (forum:Forum) -[:HAS_MEMBER]->
(p:Person {id:  6597069766845})
MATCH (forum) -[:CONTAINER_OF]->
(post:Post)
RETURN DISTINCT COUNT(DISTINCT post)
AS
JoinedForumPostCount
```

Structure: join_heavy, aggregation.

Relationships: CONTAINER_OF, HAS_MEMBER.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {JoinedForumPostCount: 1}; observed rows: 1

11. **SNB / Complex Retrieval / easy.** *Question:* Which people are members of forums containing posts tagged 'Manuel_Noriega'?

```
MATCH (forum:Forum) -[:HAS_MEMBER]->
(p:Person), (forum)
-[:CONTAINER_OF]->
(post:Post) -[:HAS_TAG]-> (tag:Tag
{name:  'Manuel_Noriega'})
RETURN DISTINCT p.id AS PersonId
LIMIT 200
```

Structure: join_heavy, bounded_result.

Relationships: CONTAINER_OF, HAS_MEMBER, HAS_TAG.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {PersonId: 4398046511220}; observed rows: 19

12. **SNB / Negation Difference / medium.** *Question:* Which members of forum 'Wall of Celso Oliveira' have not liked any post?

```
MATCH (forum:Forum {title: 'Wall of
Celso Oliveira'}) -[:HAS_MEMBER]->
(p:Person)
WHERE NOT (p) -[:LIKES]-> (:Post)
RETURN DISTINCT p.id AS PersonId,
p.firstName AS FirstName, p.lastName
AS
LastName
LIMIT 200
```

Structure: join_heavy, negation, bounded_result.

Relationships: HAS_MEMBER, LIKES.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {FirstName: K., LastName: Sen, PersonId: 94}; observed rows: 1

13. **SNB / Path Temporal / easy.** *Question:* Which people are within two knows hops of person with id 4398046511136?

```
MATCH (src:Person {id:
4398046511136})
-[:KNOWS*1..2]-> (dst:Person)
RETURN DISTINCT dst.id AS PersonId,
dst.firstName AS FirstName,
dst.lastName
AS LastName
LIMIT 200
```

Structure: single_hop, path, bounded_result.

Relationships: KNOWS.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {FirstName: Rafael, LastName: Fernández, PersonId: 4398046511333}; observed rows: 25

14. **SNB / Ranking Topk / medium.** *Question:* Among forums tagged 'James_K._Polk', which forum has the most members?

```
MATCH (forum:Forum) -[:HAS_TAG]->
(tag:Tag {name: 'James_K._Polk'})
MATCH (forum) -[:HAS_MEMBER]->
(person:Person)
WITH forum, COUNT(DISTINCT person)
AS
memberCount
RETURN DISTINCT forum.title AS
ForumTitle, memberCount
ORDER BY memberCount DESC
LIMIT 1
```

Structure: join_heavy, aggregation, order_rank, bounded_result.

Relationships: HAS_MEMBER, HAS_TAG.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {ForumTitle: Wall of Javed Khan, memberCount: 33}; observed rows: 1

15. **SNB / Simple Aggregation / easy.** *Question:* How many posts are tagged 'Vietnam'?

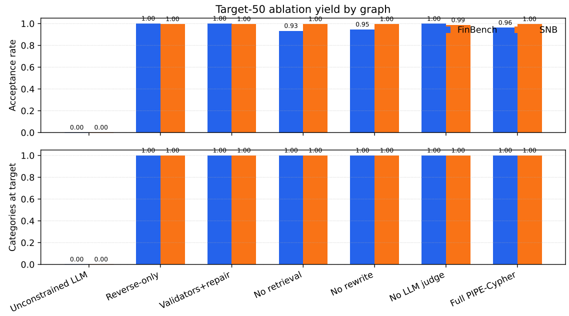

Figure 1: Audited target-50 ablation suite over FinBench and SNB. The suite reaches all graph/variant/category targets and is included as appendix evidence while the larger target-100 and seeded repeat suites continue running for stronger final reliability evidence.

```
MATCH (post:Post) -[:HAS_TAG]->
(tag:Tag
{name: 'Vietnam'})
RETURN DISTINCT COUNT(DISTINCT post)
AS
PostCount
```

Structure: single_hop, aggregation.

Relationships: HAS_TAG.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {PostCount: 1}; observed rows: 1

16. **SNB / Simple Retrieval / easy.** *Question:* Which post IDs did person with id 4398046511124 like?

```
MATCH (p:Person {id:
4398046511124})
-[:LIKES]-> (post:Post)
RETURN DISTINCT post.id AS PostId
LIMIT 200
```

Structure: single_hop, bounded_result.

Relationships: LIKES.

Gates: RO/Syn/Schema/Exec/Judge.

Result sample: {PostId: 343597385744}; observed rows: 5

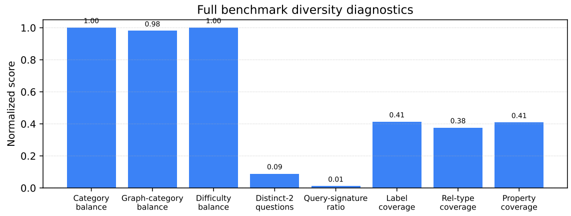

Figure 2: Diversity diagnostics for the full 3,000-example export. Category, graph-category, and difficulty balance are strong by construction; schema coverage and query-signature diversity expose remaining concentration from the fallback seeded-template run.

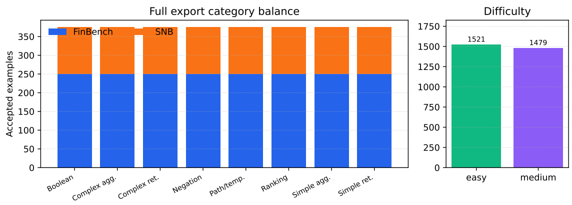

Figure 3: Full 3,000-example export distribution. The benchmark preserves the planned 2:1 FinBench/SNB graph mix while balancing all eight Cypher workload categories and maintaining a near-even easy/medium difficulty split.

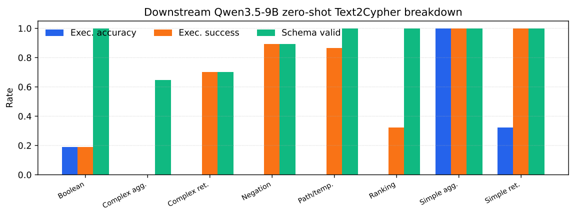

Figure 4: Downstream zero-shot Qwen3.5-9B Text2Cypher evaluation by workload category on the full test split. The breakdown separates final execution accuracy from parse/schema/execution success signals, making difficult operational categories visible.

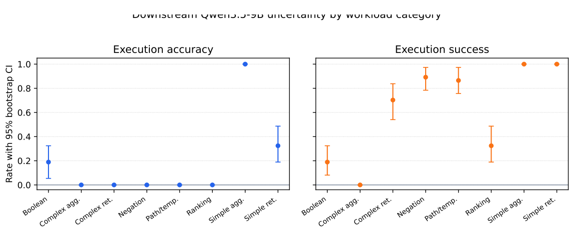

Figure 5: Bootstrap uncertainty for downstream Qwen3.5-9B Text2Cypher evaluation by workload category. Error bars show 95% row-level bootstrap intervals over the full exported test split.